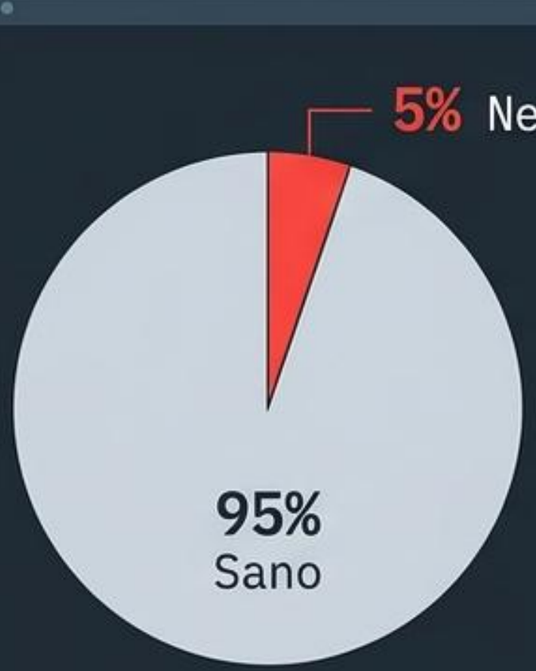
Sesión 07 (Parte A): Evaluación Rigurosa de Modelos de Deep Learning

Más allá del Accuracy: métricas que realmente importan

```
[Nivel]: Básico → Intermedio → Avanzado
[Curso]: Ingeniería del Conocimiento (ISO56B) — UNCP
[Docente]: Msc. Jaime Antonio Huaytalla Pariona

> status: ready for audit_
```

La trampa del Accuracy



Un modelo que dice 'Sano' a todo tiene 95% de accuracy... y es 100% inútil.

MODEL_LOG: PREDICCIÓN GLOBAL = 'SANO'. ACCURACY_CURRENT = 0.950.
VALIDATION_STATUS: FAILED. CRITICAL_ERROR: NO_NEUMONIA_DETECTED.



El Vigilante Nocturno:

Reporta 'todo tranquilo' cada noche sin hacer rondas.

Tiene un 99% de accuracy.

Pero la única noche del robo, falló.

SYSTEM_LOG: SECURITY_CHECK_STATUS: OK. ROUNDS_COMPLETED: 0/10.
ALERT_LEVEL: NORMAL. REPORT_GENERATED: 'TODO TRANQUILO'.
ACCURACY_METRIC: 0.99.
EVENT_LOG: CRITICAL_FAILURE - INTRUSION_DETECTED (MISSED).

En problemas desbalanceados, el Accuracy oculta fracasos catastróficos. Importa QUÉ detectamos, no solo cuánto acertamos en general.

El desbalance es la norma, no la excepción

Dominio del Problema	Clase Positiva (Rara)	Clase Negativa (Común)	Ratio Típico
Detección de fraude bancario	**Fraude**	**Legítima**	1:1000
Diagnóstico de enfermedades	**Enfermo**	**Sano**	1:200
Detección de fallas industriales	**Falla**	**Normal**	1:100
Filtro de spam	**Spam**	**Correo**	1:5
CIFAR-10 (Académico)	Una clase	Resto	1:9 (Balanceado)



Necesitamos instrumentos que auditen qué tipo de errores comete el modelo.

La Matriz de Confusión: Radiografía del Error

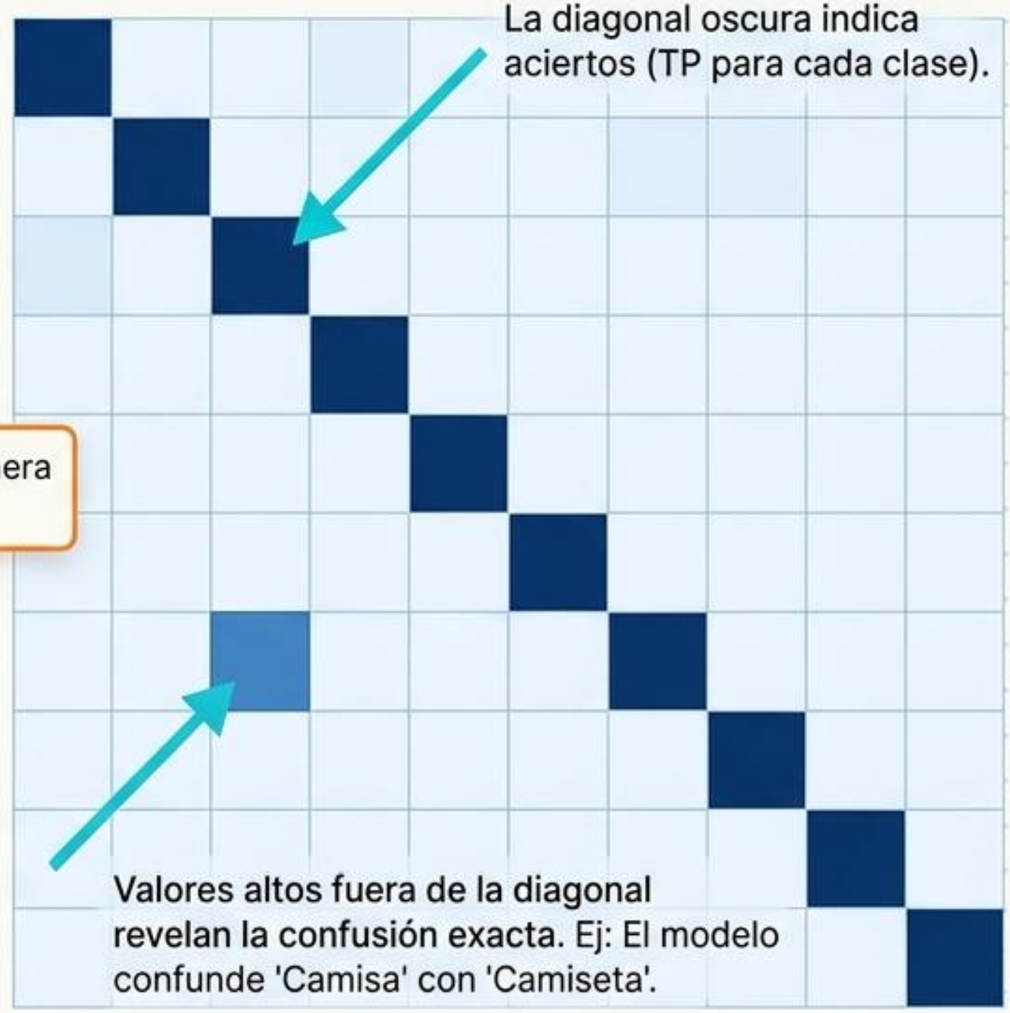
		PREDICCIÓN DEL MODELO: Positivo	PREDICCIÓN DEL MODELO: Negativo
REALIDAD: Positivo		 **Verdadero Positivo (TP)** Hay fuego, suena la alarma. ¡Perfecto!	 **Falso Negativo (FN)** Fuego invisible. ¡Peligro mortal!
REALIDAD: Negativo		 **Falso Positivo (FP)** Falsa alarma (Molesto, pero no mortal).	 **Verdadero Negativo (TN)** No hay fuego, todo tranquilo.

Implementación y Diagnóstico Multiclase

```python
from sklearn.metrics import confusion_matrix

# Comparar realidad vs predicción
cm = confusion_matrix(y_test, y_pred)
```

Para N clases, genera una matriz N×N.



Precision (Precisión): La calidad de tus alertas

The Fisherman Analogy



Dije que atrapé 10 peces grandes.
8 lo eran, 2 eran basura. Mi precisión es 80%.

De todos los que el modelo marcó como positivos, ¿cuántos REALMENTE lo eran?

Math & Cost

```
Precision = TP / (TP + FP)
```

⚠ **Optimiza Precision** cuando el costo de un Falso Positivo (FP) es inaceptable.



Bloquear un correo vital de tu jefe.
(Spam filter)



Encarcelar a una persona inocente.
(Sistema judicial)

Recall (Sensibilidad) y el Dilema del Umbral

The Metal Detector Analogy

Subtite on the metal detector analogy

$$\text{Recall} = \frac{TP}{TP + FN}$$



¿De los positivos reales, cuántos logré encontrar?

Optimiza Recall cuando un **Falso Negativo (FN)** es fatal (ej. Cáncer, fallas de puentes).

El dilema Precision vs Recall: No se puede tener todo

Portero Estricto



Revisa a fondo. Nadie malo entra (**Alta Precision**), pero fans legítimos se quedan fuera de la fila lenta (**Bajo Recall**).

Portero Permisivo



Todos entran rápido y nadie se queda fuera (**Alto Recall**), pero se cuelan personas sin entrada (**Baja Precision**).

F1-Score: El balance armónico

F1-Score es la **media armónica**. Penaliza severamente si **Precision** o **Recall** caen a cero. Es la **métrica ideal para clases desbalanceadas**.

```
from sklearn.metrics import classification_report

              precision    recall  f1-score   support

   Camiseta       0.82      0.86      0.84      1000
   Pantalón       0.98      0.97      0.97      1000
        ...        ...       ...       ...       ...
     Camisa       0.70      0.63      0.66      1000
  Zapatilla       0.95      0.95      0.95      1000
  macro avg       0.88      0.88      0.88     10000
```

Muestra la cantidad real de ejemplos por clase. Útil para detectar desbalance.

La clase más difícil para el modelo. F1=0.66.

Variantes Multiclase: Eligiendo el promedio correcto

Variante (Parámetro)	Cómo se calcula	Cuándo usarlo (Regla de oro)
`average='macro'`	Promedio simple de todas las clases.	Trata a todas las clases como igualmente importantes, sin importar su tamaño.
`average='weighted'`	Promedio ponderado por la cantidad de muestras (support).	Cuando quieres que el puntaje refleje la distribución real de las clases.
`average='micro'`	Calcula TP, FP, FN globales primero.	Equivale al Accuracy general. Útil solo en desbalance extremo.

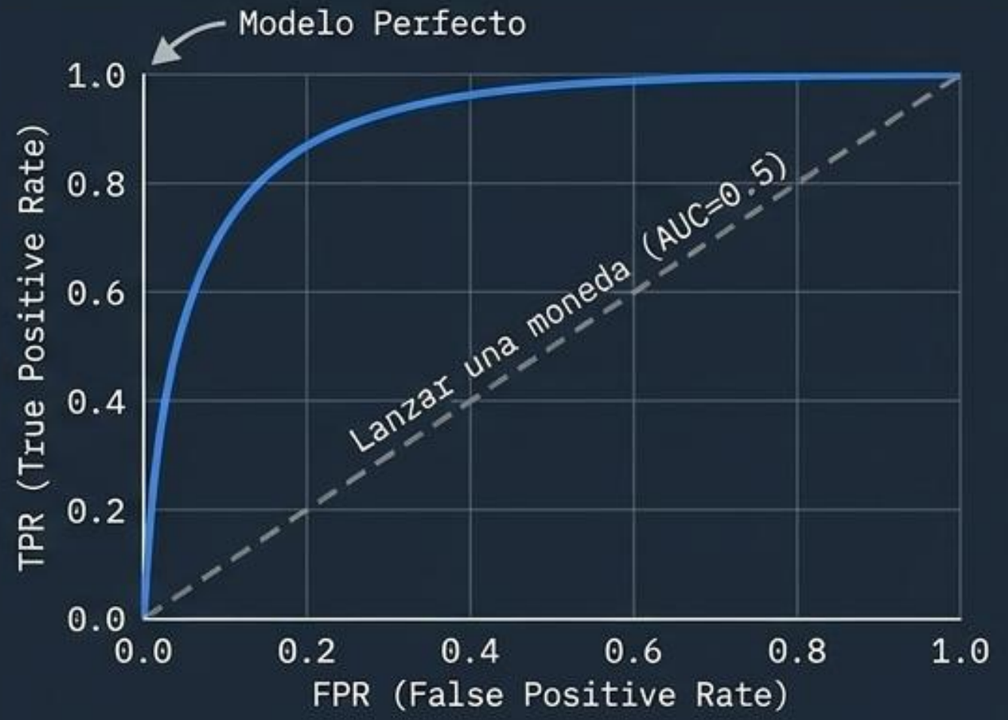
```
from sklearn.metrics import f1_score

f1_macro = f1_score(y_test, y_pred, average='macro')
f1_weight = f1_score(y_test, y_pred, average='weighted')
```

Curvas ROC y AUC: Evaluando umbrales dinámicos



La curva ROC muestra qué pasa al girar el umbral. ¿Muy sensible? Detectas todo el fuego (Alto TPR) pero con muchas falsas alarmas de la cocina (Alto FPR).



Mini diagnóstico:

✅ 0.90 - 1.00:	Excelente
0.80 - 0.90:	Bueno
⚠️ 0.50 - 0.60:	Falla (Cercano al azar)

Implementación ROC Multiclase y Curvas P-R

```
# Binarizar para multiclase (One-vs-Rest)
y_test_bin = label_binarize(y_test,
classes=range(10))

# Curva para una clase específica
fpr, tpr, _ = roc_curve(y_test_bin[:, i],
y_pred_probs[:, i])
roc_auc = auc(fpr, tpr)
```

⚠️ La curva ROC es engañosamente optimista con desbalance extremo porque los Verdaderos Negativos (TN) inflan el cálculo.

Clases Balanceadas	Usar ROC Curve ✅
Clases Muy Desbalanceadas	Usar Precision-Recall Curve ✅ (Se enfoca directamente en la clase minoritaria).

Validación Cruzada (K-Fold): Más allá del split único



Un solo split Train/Test es como evaluar a un alumno con un solo examen. Su nota dependerá de la suerte de qué preguntas tocaron.

Fold 1	[VAL]	[Train]	[Train]	[Train]	[Train]
Fold 2	[Train]	[VAL]	[Train]	[Train]	[Train]
Fold 3	[Train]	[Train]	[VAL]	[Train]	[Train]
Fold 4	[Train]	[Train]	[Train]	[VAL]	[Train]
Fold 5	[Train]	[Train]	[Train]	[Train]	[VAL]

Promediamos 5 evaluaciones independientes para obtener el rendimiento real (Media ± Desviación Estándar).

K-Fold en Deep Learning: Consideraciones Prácticas

```
for train_idx, val_idx in kfold.split(X_train):
    # ⚠️ CRÍTICO: Crear modelo NUEVO en cada fold
    # Si no, estarás reentrenando con datos vistos
    model = crear_modelo_cnn()
    model.fit(X_train[train_idx], y_train[
    train_idx])
```

Aspecto	Machine Learning (sklearn)	Deep Learning (Keras)
K típico	5 o 10	3 o 5 (Por costo computacional)
Tiempo	Segundos	Minutos a Horas
Varianza	Baja	Alta (Por la inicialización aleatoria de pesos)
Solución Desbalance	`StratifiedKFold`	`StratifiedKFold`

Protocolo de Comparación Justa

1. Mismos Datos: Misma semilla (random_state), mismo split, mismo preprocesamiento.

2. Mismas Condiciones: Mismas épocas, misma arquitectura base, mismo data augmentation.

3. Mismas Métricas: Evaluar todos bajo la misma lupa (F1, AUC, etc.).

Modelo	Parámetros	Accuracy (%)	F1 macro	Tiempo/época (s)
CNN Simple	62,410	85.2	0.851	12
CNN + BatchNorm	62,794	87.8	0.877	14
ResNet-20	274,442	91.3	0.912	28

No elijas el mejor modelo solo por Accuracy. Considera el costo computacional (Tiempo) y el tamaño (Parámetros).

Resumen de Auditoría y Diagnóstico

Mapa de Decisión (Flowchart)



The Inspector's Checklist

- [x] ¿Normalicé los datos después de dividir (evitar Data Leakage)?
- [x] ¿Fijé las semillas aleatorias para reproducibilidad?
- [x] ¿Verifiqué la matriz de confusión por clase?
- [x] ¿Elegí la métrica según el costo real del error en el dominio?
- [x] ¿Usé un test set completamente aislado?

Próximo paso — **Continúa en Parte B: Interpretabilidad y Explicabilidad** (Abriendo la Caja Negra)